Assignment 1: Prepare a test strategy document based on the domain, using a chain-of-thought approach.

Instructions - Generate testcases for NEFT transfer within same bank account.

-[STRICTLY] Generate Functional testcases with positive and negative

-[MANDATORY] Include Testcase Name, description, expected result, test data, Precondition, Test Step, Priority and severity for each testcases.

-[CRITICAL] --List the testcases in priority and severity order

-[MANDATORY] --Generate Only 5 testcases based on priority and severity.

- [IMPORTANT] - ask me 3 questions one by one and proceed only if i answer for the questions one by one

Context -

Act like a Banking Domain Assistant expertise in functional testing

Example -

TC-ID -TC001

Testcase Name - Testcase Name 1
Priority-P1

Severity - S1

Test Description --To verify whether fund transfer is successful

Precondition --Application should be up and running, Accounts should have sufficient fund

Teststep --Login to APP. Enter the account number for fund transfer .3 Transfer Funds

Expected Result --Fund transfer should be successful

Test data -- Account 1- 445556 Account 2- 434343 Transfer amount -1000$

Tone -

[STRICTLY] Professional tone in review comments and summary

Output -

Display the results in table format

Persona -

Act as a Team Lead to review the testcases and give the review comments

Before generating the test cases, I will gather required inputs step by step.

Question 1:

What type of channel is being used for the NEFT transfer?

Options (please specify one):

1. Mobile Banking Application
2. Internet Banking (Web)
3. Branch Teller System

Please provide your answer so I can proceed to the next question.

Internet Banking (Web)

Question 2:

Should the beneficiary already be **pre-registered within the same bank**, or should test cases include **adding a new beneficiary as part of the flow**?

Options (please specify one):

1. Pre-registered beneficiary only
2. Include new beneficiary addition flow

Please provide your answer to proceed.

Pre-registered beneficiary only

Question 3:

Should NEFT transactions be tested considering **transaction limits** (e.g., minimum/maximum transfer amount restrictions)?

Options (please specify one):

1. Yes – include transaction limit validations
2. No – exclude limit validations

Please provide your answer so I can proceed with generating the test cases.

Yes – include transaction limit validations

Based on your inputs, here are the **functional test cases for NEFT transfer within the same bank (Internet Banking – Pre-registered beneficiary – including transaction limits)**, ordered by **Priority and Severity**.

Test Cases Table

TC-ID	Testcase Name	Priority	Severity	Test Description	Precondition	Test Steps	Test Data	Expected Result
TC001	Successful NEFT Transfer Within Same Bank (Valid Limits)	P1	S1	Verify NEFT transfer is successful with valid beneficiary and amount within transaction limits	Application is up; User logged in to Internet Banking; Beneficiary pre-registered; Sufficient balance; Amount within limits	1. Login to Internet Banking 2. Navigate to NEFT transfer 3. Select pre-registered beneficiary 4. Enter valid amount within limits 5. Confirm transfer	Source A/C: 1234567890 Beneficiary A/C: 0987654321 Amount: 10,000 Limit: 1–1,00,000	Transfer should be successful and confirmation message with transaction reference number displayed
TC002	NEFT Transfer Failure Due to Insufficient Balance	P1	S1	Verify system prevents transfer when balance is lower than transfer amount	Application is up; User logged in; Beneficiary pre-registered; Insufficient balance	1. Login 2. Navigate to NEFT transfer 3. Select beneficiary 4. Enter amount greater than available balance 5. Initiate transfer	Balance: 2,000 Transfer Amount: 5,000	Transfer should be declined with "Insufficient Balance" error and no debit should occur
TC003	NEFT Transfer Exceeding	P1	S2	Verify system restricts	Application is up; User logged in;	1. Login 2. Navigate	Amount: 2,00,000 Max	System should block

TC-ID	Testcase Name	Priority	Severity	Test Description	Precondition	Test Steps	Test Data	Expected Result
	Maximum Limit			transfer when amount exceeds maximum NEFT limit	Beneficiary pre-registered	to NEFT transfer 3. Enter amount exceeding max limit 4. Submit transfer	Limit: 1,00,000	transaction and display "Exceeds Maximum Limit" message
TC004	NEFT Transfer Below Minimum Limit	P2	S2	Verify system restricts transfer when amount is below minimum NEFT limit	Application is up; User logged in; Beneficiary pre-registered	1. Login 2. Navigate to NEFT transfer 3. Enter amount below minimum limit 4. Submit transfer	Amount: 0 or 0.50 Min Limit: 1	System should display validation error and prevent transaction
TC005	NEFT Transfer Cancellation Before Final Confirmation	P3	S3	Verify user can cancel NEFT transfer before final submission	Application is up; User logged in; Beneficiary pre-registered	1. Login 2. Navigate to NEFT transfer 3. Enter valid details 4. Click cancel before confirmation	Amount: 5,000	Transaction should be cancelled and no amount should be debited

---

Review Comments (Team Lead Perspective)

- The test cases demonstrate **clear coverage of critical transaction flows**, especially high-risk scenarios such as insufficient balance and limit validations.
- Proper inclusion of both **positive (TC001)** and **negative scenarios (TC002–TC004)** ensures functional robustness.
- Priority (P1–P3) and Severity (S1–S3) are appropriately aligned with **business impact and risk exposure**.
- The inclusion of **transaction limit validations** strengthens compliance with banking rules.
- Test steps are concise, reproducible, and aligned with Internet Banking workflows.

Suggestion for Improvement:

- Consider adding **audit trail verification** and **transaction status tracking (success/pending/failure)** in future test cycles.
- Additional coverage for **session timeout during transaction** and **duplicate transaction prevention** would enhance completeness.

Overall, the test suite is **well-structured, risk-focused, and suitable for functional validation in a banking environment**.

Assignment 2: Design the automation framework structure using a chain-of-thought approach

Instructions:

- Generate a Selenium Java based test script.

- Verify the login in gmail in chrome, edge browser.

- Use explicit waits instead of Thread.sleep().

- Validate both successful and unsuccessful login attempts.

-[MANDATORY] Only TestNG

-[MANDATORY] Use only @BeforeMethod and @Aftermethod in TestNg Annotations.

-[STRICT] - ask me 3 questions and implement the framework after i answer each question and then proceed.

Context:

Act like a Automation Assistant with gmail login knowledge

Tone:

[STRICTLY] Professional tone in review comments and summary

Output:

only code without any markdown or extra text Persona Act as a Test Automation Architect to review the testcases and give the review comments

Before proceeding with the framework implementation, I need to gather required inputs.

Question 1:
How do you want to manage browser execution?

Options:

1. Use TestNG parameters (browser passed from XML)
2. Use WebDriverManager with dynamic browser selection in code
3. Hardcode both Chrome and Edge execution within test methods

Please provide your answer so I can proceed to the next question.

Use TestNG parameters (browser passed from XML)

Question 2:
How should test data (credentials) be handled?

Options:

1. Hardcoded داخل the test script
2. Use TestNG @Parameters (pass from XML)
3. Read from external file (e.g., Excel/Properties)

Please provide your answer to proceed.

Use TestNG @Parameters (pass from XML)

Question 3:
What level of framework structure do you expect?

Options:

1. Simple single class (no Page Object Model)
2. Page Object Model (POM) with separate page classes
3. Hybrid framework (POM + utilities + base classes)

Please provide your answer so I can proceed with the implementation.

Page Object Model (POM) with separate page classes

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.edge.EdgeDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.testng.Assert;
import org.testng.annotations.*;

import java.time.Duration;

// Base Test Class
class BaseTest {
protected WebDriver driver;
protected WebDriverWait wait;

@Parameters({"browser"})
@BeforeMethod
public void setUp(String browser) {
    if (browser.equalsIgnoreCase("chrome")) {
        driver = new ChromeDriver();
    } else if (browser.equalsIgnoreCase("edge")) {
        driver = new EdgeDriver();
    }
    driver.manage().window().maximize();
    driver.get("https://accounts.google.com/");
    wait = new WebDriverWait(driver, Duration.ofSeconds(15));
}

@AfterMethod
public void tearDown() {
    if (driver != null) {
        driver.quit();
    }
}

}

// Page Object Class
class LoginPage {
WebDriver driver;
WebDriverWait wait;

By emailField = By.id("identifierId");
By emailNext = By.id("identifierNext");
```

```
By passwordField = By.name("password");
By passwordNext = By.id("passwordNext");
By composeBtn = By.xpath("//div[text()='Compose']");
By errorMsg = By.xpath("//div[contains(text(),'Couldn't find your Google Account')]");

public LoginPage(WebDriver driver, WebDriverWait wait) {
    this.driver = driver;
    this.wait = wait;
}

public void enterEmail(String email) {
    WebElement element =
wait.until(ExpectedConditions.visibilityOfElementLocated(emailField));
    element.sendKeys(email);
}

public void clickEmailNext() {
    wait.until(ExpectedConditions.elementToBeClickable(emailNext)).click();
}

public void enterPassword(String password) {
    WebElement element =
wait.until(ExpectedConditions.visibilityOfElementLocated(passwordField));
    element.sendKeys(password);
}

public void clickPasswordNext() {
    wait.until(ExpectedConditions.elementToBeClickable(passwordNext)).click();
}

public boolean isLoginSuccessful() {
    return
wait.until(ExpectedConditions.visibilityOfElementLocated(composeBtn)).isDisplayed();
}

public boolean isErrorDisplayed() {
    return
wait.until(ExpectedConditions.visibilityOfElementLocated(errorMsg)).isDisplayed();
}

}

// Test Class
public class GmailLoginTest extends BaseTest {
```

```
@Test
@Parameters({"validEmail", "validPassword"})
public void verifySuccessfulLogin(String email, String password) {
    LoginPage loginPage = new LoginPage(driver, wait);

    loginPage.enterEmail(email);
    loginPage.clickEmailNext();
    loginPage.enterPassword(password);
    loginPage.clickPasswordNext();

    Assert.assertTrue(loginPage.isLoginSuccessful(), "Login failed for valid credentials");
}

@Test
@Parameters({"invalidEmail"})
public void verifyUnsuccessfulLogin(String email) {
    LoginPage loginPage = new LoginPage(driver, wait);

    loginPage.enterEmail(email);
    loginPage.clickEmailNext();

    Assert.assertTrue(loginPage.isErrorDisplayed(), "Error message not displayed for invalid
login");
}

}
```